

Quantum Triangle Finding via Triangle-Free Templates and Density-Sensitive Search

Shiva Kintali*

September 20, 2026

Abstract

We give a bounded-error quantum algorithm that finds a triangle in an unknown unweighted n -vertex graph, or reports that none exists, using

$$O\left(n^{5/4} \exp(-c \log_2^* n)\right)$$

adjacency queries for an absolute constant $c > 0$, improving the $O(n^{5/4})$ bound by an unbounded subpolynomial factor. The algorithm combines paid learning of a triangle-free template with a density-sensitive extended learning graph requiring sparsity only on negative inputs. Quantitative approximate-homomorphism bounds yield the saving, and geometric self-reduction recovers a witness without logarithmic overhead. All graph access is charged; non-query computation is unrestricted.

1 Introduction

Triangle finding asks whether an unknown simple, undirected, unweighted n -vertex graph contains three distinct pairwise adjacent vertices and, if so, asks for such a triple. In the quantum adjacency-query model, an oracle returns edge-indicator bits and may be queried in superposition. Positive certificates consist of just three edges, but searching over vertex triples alone does not exploit the overlap among these certificates.

Prior work. Early algorithms used amplitude amplification and quantum walks inspired by element distinctness [1, 2, 3]. Learning graphs gave bounds of $O(n^{35/27})$ [12] and then $O(n^{9/7})$ [4]. Belovs and Rosmanis established a nearly matching limitation for the triangle certificate structure in the non-adaptive learning-graph model [5]; this is not a lower bound for unrestricted quantum algorithms. Le Gall obtained $\tilde{O}(n^{5/4})$ queries using a combinatorial decomposition and quantum walks [8]. Le Gall and Nakajima improved the bound under edge-sparsity promises [9]. Carette–Laurière–Magniez (CLM) introduced extended learning graphs and obtained $O(n^{5/4})$ queries without logarithmic overhead [7]. Graph collision [3] and nested quantum walks [6] provide related ingredients.¹

*Email: shiva.kintali@gmail.com Homepage: shivakintali.github.io

¹Triangle-sum algorithms under an at-most-one-target promise [10] and algorithms for edge-list oracles [11] concern different promises or access models.

Result and contributions. Theorem 1 gives an algorithm that outputs a triangle or correctly reports its absence with probability at least $2/3$, using $O(n^{5/4} \exp(-c \log_2^* n))$ queries for an absolute constant $c > 0$. Here $\log_2^* n$ counts binary-logarithm iterations until the value is at most one. The saving is unbounded but subpolynomial: the bound is $o(n^{5/4})$, not a fixed-power improvement. The gap to the $\Omega(n)$ lower bound [3] remains.

Our method extends CLM’s extended-learning-graph framework, sparse-loading paths, and nested triangle-search decomposition [7]. Our additional ingredients are a density-dependent loading offset and asymmetric tripartite parameters, yielding a search primitive that requires sparsity only on negative inputs. We combine this primitive with paid learning of a triangle-free template whose mask intersects every triangle and, with high probability on triangle-free inputs, contains few graph edges. The saving comes from this combination, rather than a new learning-graph model or a new approximate-homomorphism theorem.

For a tripartite graph with three n -vertex parts, the resulting primitive uses

$$O\left(n^{5/4}[\delta \log(2 + 1/\delta)]^{1/8} + n^{6/5}\right)$$

queries for sufficiently small positive δ , with a constant independent of δ . Crucially, only triangle-free inputs must have at most δn^2 edges in the designated relation; all inputs containing a triangle are included in the promise (Lemma 2). The $n^{6/5}$ term is part of this upper bound, not a lower bound for triangle finding.

The structural input is the triangle-free approximate-homomorphism theorem in the formulation of Fox–Zhao [13, Theorem 1.6]. Its quantitative bound uses Hoppen–Kohayakawa–Lang–Lefmann–Stagni [15] and Fox’s graph-removal bound [14]. Learning graph partitions from adjacency samples has classical precedents [16]; we do not claim template learning itself as new. Our contribution combines paid template selection with the negative-input-only sparsity guarantee above. No template, edge-density promise, or improved graph-collision algorithm is assumed. As in the non-uniform query model specified in Section 2, all graph access is charged, but non-query computation is unrestricted; no efficient running-time bound is asserted.

Proof strategy. A template consists of a map $\phi : [n] \rightarrow V(H)$ to a loopless triangle-free graph H . Its mask consists of vertex pairs whose images are nonadjacent in H . Every triangle has an edge in this mask: otherwise its images would form a triangle in H . We query random pairs and choose a template minimizing the sampled density of graph edges in its mask. The structural theorem and uniform concentration make the true masked-edge density small on triangle-free inputs, with high probability. On positive inputs, coverage holds for every template, without a concentration assumption. The masked relation therefore fits the asymmetric promise of Lemma 2. For sufficiently large n , choosing template tower height $\log_2^* n - 3$ gives the stated saving. A four-child geometric self-reduction recovers a witness without logarithmic query overhead.

To prove the primitive, the density-offset loader keeps negative complexity linear in the number of loaded ones plus an offset, with a logarithmic factor depending on δ rather than the input size. Its positive complexity is bounded uniformly on every input. Applying it to the outer cache and terminal check of an asymmetric tripartite learning graph, then averaging exactly and balancing the parameters, yields the density exponent $1/8$.

Organization. Section 2 states the main theorem and its two ingredients. Section 3 derives unrestricted triangle finding from the sparse-relation lemma, including paid template learning, the quantitative saving, and witness recovery. Sections 4 and 5 establish the density-offset loader and the sparse-relation lemma, respectively.

2 Main theorem and proof architecture

The input is a simple, undirected, unweighted graph G on $[n] = \{1, \dots, n\}$, where $n \geq 1$, with the bit oracle

$$O_G|u, v, z\rangle = |u, v, z \oplus G_{uv}\rangle, \quad u, v \in [n], \quad z \in \{0, 1\}.$$

Here $G_{uv} = G_{vu}$ and $G_{uu} = 0$, and the oracle acts linearly on superpositions. We count oracle uses, not classical or quantum gate complexity. We use the non-uniform query model: for each input length, the algorithm's input-independent operations may be fixed in advance, but access to the unknown graph is only through its oracle. In particular, the finite optimization below may take enormous time. The algorithm is not given a template, a density estimate, or an input-dependent certificate. Any such information used below is obtained from charged queries. The notation $\log_2^* n$ denotes the number of binary-logarithm iterations needed to reach a value at most one; unsubscripted $\log$ and $\ln$ both denote the natural logarithm.

Theorem 1. *Let $C_3 \geq 1$ be any fixed constant satisfying (2). For every fixed $0 < c' < \ln 2/(8C_3)$ there is a quantum algorithm that, on every n -vertex input graph, outputs three distinct vertices forming a triangle, or correctly reports that none exists, with probability at least $2/3$. Its worst-case number of adjacency queries is*

$$O\left(n^{5/4} \exp(-c' \log_2^* n)\right),$$

where the implicit constant may depend on C_3 and c' , but not on n or the input graph.

Section 3.2 sets the density offset δ_n and an integer template-size bound using only n and the fixed constant C_3 . Only an upper bound for the extremal template size is needed, not its exact value; the template itself is learned using charged queries. An admissible C_3 may be replaced by any larger constant, including an integer, since (2) persists under this replacement. Smaller admissible values certify a larger range of rates c' . All parameter choices below apply for sufficiently large n ; the finitely many smaller input sizes can be handled by querying the entire graph and enlarging the implicit constant.

2.1 The sparse-relation quantum primitive

Let U, V, W be disjoint copies of $[n]$, and let $R \subseteq U \times W$, $B \subseteq V \times W$, and $A \subseteq U \times V$ be three arbitrary Boolean relations. Thus n is the size of each part, not the total number of vertices. A query specifies a relation and a pair of its indices and returns the corresponding bit; these query labels may be in superposition. The triangle predicate is

$$\bigvee_{u \in U, v \in V, w \in W} R_{uw} B_{vw} A_{uv}.$$

Positive and negative inputs mean inputs on which this predicate is one and zero, respectively.

Fix once and for all an absolute numerical constant $C_2 \geq 1$, independent of n and δ . For $0 < \delta \leq 1$, write

$$L = \log(2 + 1/\delta), \quad \rho = C_2 \delta L.$$

The function $\delta \mapsto \delta \log(2 + 1/\delta)$ is continuous and strictly increasing on $(0, 1]$, because its derivative is

$$\log(2 + 1/\delta) - \frac{1}{1 + 2\delta} \geq \ln 3 - 1 > 0.$$

It tends to zero as $\delta \downarrow 0$, whereas its value at 1 is $\ln 3 > 1$. Consequently there is a unique $\delta_0 \in (0, 1)$ satisfying $C_2 \delta_0 \log(2 + 1/\delta_0) = 1$, and $\rho \leq 1$ precisely when $\delta \leq \delta_0$.

Lemma 2. *There are constants $C > 0$ and $n_0 \geq 1$, depending only on the fixed C_2 , such that for every integer $n \geq n_0$ and every known parameter $0 < \delta \leq \delta_0$ there is a quantum decision algorithm for the triangle predicate with the following guarantees. Its promise domain consists of all positive inputs and all negative inputs with $|R| \leq \delta n^2$. On this domain it is correct with probability at least $2/3$, and on every input it uses at most*

$$Cn^{5/4} \max\{\rho, n^{-2/5}\}^{1/8} \quad (1)$$

queries. In particular, this cap is

$$CC_2^{1/8} n^{5/4} [\delta \log(2 + 1/\delta)]^{1/8} \quad \text{if } n^{-2/5} \leq \rho \leq 1,$$

and $Cn^{6/5}$ if $\rho < n^{-2/5}$. Both C and n_0 are independent of n and δ . No correctness guarantee is asserted outside the promise domain.

Only negative inputs must have a sparse designated relation; neither A nor B is required to be sparse, and every positive input is included. The parameter δ is chosen in advance, not obtained from an uncharged estimate of $|R|$. Section 5 proves the lemma, including the uniformity in δ and the fixed query cap.

2.2 The triangle-free template theorem

The triangle-free lemma, in the formulation of Fox–Zhao [13, Theorem 1.6], states that for every $\epsilon > 0$ there is a positive integer $M(\epsilon)$ with the following property. For every n and every triangle-free n -vertex graph G , there are a loopless triangle-free graph H on at most $M(\epsilon)$ vertices and a map $\phi : V(G) \rightarrow V(H)$ such that at most ϵn^2 *unordered edges* of G map to nonedges of H . The map need not be injective: an edge whose endpoints have the same image counts as an error, because H has no loops. There is no condition on the images of nonedges of G .

Let $M(\epsilon)$ denote the least positive integer with this property, uniformly over n and G . The bounds of Hoppen–Kohayakawa–Lang–Lefmann–Stagni [15, Section 3.3] and Fox [14, Theorem 1], in the formulation of [13, inequality (1.1)], imply that an absolute constant $C_3 \geq 1$ satisfies

$$M(\delta/8) \leq \text{twr}(\lceil C_3 \log_2(1/\delta) + C_3 \rceil), \quad 0 < \delta \leq 1, \quad (2)$$

where $\text{twr}(0) = 1$ and $\text{twr}(j+1) = 2^{\text{twr}(j)}$ for integers $j \geq 0$. To derive (2), let $d_{\text{TRL}}(t)$ be the supremum of $d \in (0, 1]$ such that every n -vertex graph with fewer than dn^3 triangles can be made triangle-free by deleting at most tn^2 edges. This supremum is positive and admissible because the triangle threshold is strict: if a graph has triangle density less than the supremum, it also has density less than some admissible value of d . The weak-regularity bound of [15, Section 3.3], as formulated in [13, inequality (1.1)], and Fox’s removal bound [14, Theorem 1] give absolute constants $A, C, a \geq 1$ such that

$$M(\epsilon) \leq \exp(A d_{\text{TRL}}(\epsilon/C)^{-2}), \quad d_{\text{TRL}}(t)^{-1} \leq \text{twr}(\lceil a \log_2(1/t) + a \rceil) \quad (0 < \epsilon, t < 1).$$

The constants absorb the cap at one, triangle-counting conventions, the change of logarithm base, and rounding of tower heights. Set $\ell = \lceil a \log_2(C/\epsilon) + a \rceil$. Since $\text{twr}(j)^2 \leq \text{twr}(j+1)$ for $j \geq 1$ and $A \log_2(e) u \leq 2^u$ for sufficiently large u , there is an absolute integer k such that $M(\epsilon) \leq \exp(A \text{twr}(\ell)^2) \leq \text{twr}(\ell+k)$: squaring and exponentiation add at most three levels for large ℓ , and increasing k covers the finitely many smaller heights. Thus, for an absolute $C_0 \geq 1$,

$$M(\epsilon) \leq \text{twr}(\lceil C_0 \log_2(1/\epsilon) + C_0 \rceil).$$

Substituting $\epsilon = \delta/8$ and taking $C_3 = 4C_0$ proves (2) for $0 < \delta \leq 1$. We fix one such C_3 throughout. The structural theorem is an existence statement; it does not supply a template without graph queries.

How the ingredients fit together. The template theorem supplies a small-loss candidate on each triangle-free graph. Section 3.1 uses paid empirical-risk minimization over a finite class to select a template. Every candidate yields a mask that hits every triangle, regardless of its loss. Consequently the derived tripartite instance lies in the domain of Lemma 2 deterministically on positive inputs, and with high probability on negative inputs. On the latter inputs, failure of the concentration event is included in the error budget; the query cap still holds. Section 3 proves Theorem 1 from the sparse-relation lemma and the structural input, including the parameter choice and witness recovery. Sections 4 and 5 establish the lemma independently, without using the template theorem or the main theorem.

3 From sparse-relation search to unrestricted triangle finding

Throughout this section, assume Lemma 2. We first pay for learning a template and verify the resulting promise on both signs of input. We then choose the parameters to obtain the saving and recover a witness without logarithmic query overhead. Write $Q_{\Delta}^{\text{dec}}(n)$ for the bounded-error quantum query complexity of deciding triangle existence on unrestricted n -vertex graphs.

3.1 Paid template learning and triangle coverage

Fix $0 < \delta \leq \delta_0$ and take $n \geq n_0$ as in Lemma 2. Apply the triangle-free template theorem with $\epsilon = \delta/8$. Set

$$M = \text{twr}(\lceil C_3 \log_2(1/\delta) + C_3 \rceil),$$

which bounds $M(\delta/8)$ by (2). Padding a target with isolated vertices preserves its properties. Consider the finite class

$$\mathcal{C} = \{(\phi, H) : \phi : [n] \rightarrow [M], H \text{ a simple triangle-free graph on } [M]\}.$$

It has cardinality at most $M^n 2^{\binom{M}{2}}$. For a candidate $c = (\phi, H)$, define its known mask and population loss by

$$F_c(u, v) = 1 - H_{\phi(u), \phi(v)}, \quad \ell_G(c) = \frac{1}{n^2} \sum_{u, v \in [n]} G_{uv} F_c(u, v).$$

The loss uses ordered pairs, including the zero diagonal. Every triangle in G has an edge in this mask for every $c \in \mathcal{C}$. Indeed, if all three edges avoided the mask, their images would be pairwise adjacent in H . The images would be distinct, since H has no loops, and would form a forbidden triangle.

Fix $0 < \eta < 1$. Sample q independent uniform ordered pairs $(a_i, b_i) \in [n]^2$, with replacement, and query their adjacency bits. Define

$$\widehat{\ell}(c) = \frac{1}{q} \sum_{i=1}^q G_{a_i b_i} F_c(a_i, b_i),$$

and choose a minimizer $\widehat{c}$ over $\mathcal{C}$, with fixed tie-breaking. The queried data suffice to evaluate every candidate; this finite optimization uses no additional graph queries. For each fixed candidate, Hoeffding's inequality gives a deviation probability at most $2 \exp(-q\delta^2/8)$ at tolerance $\delta/4$. A union bound therefore yields

$$\Pr\left(\sup_{c \in \mathcal{C}} |\widehat{\ell}(c) - \ell_G(c)| > \delta/4\right) \leq \eta$$

for

$$q = \left\lceil \frac{8}{\delta^2} \left(n \ln M + \binom{M}{2} \ln 2 + \ln(2/\eta) \right) \right\rceil. \quad (3)$$

No independence between different candidates is required.

If G is triangle-free, the template theorem supplies $c_* \in \mathcal{C}$ with at most $(\delta/8)n^2$ bad *unordered* edges, hence $\ell_G(c_*) \leq \delta/4$. On the simultaneous concentration event,

$$\ell_G(\hat{c}) \leq \hat{\ell}(\hat{c}) + \delta/4 \leq \hat{\ell}(c_*) + \delta/4 \leq \ell_G(c_*) + \delta/2 \leq 3\delta/4.$$

Thus the relation $R_{uw} = G_{uw}F_{\hat{c}}(u, w)$ has at most δn^2 ones. On a positive input no such density bound is needed: the mask still hits every triangle.

Form three labeled copies U, V, W of the original vertex set, with

$$R_{uw} = G_{uw}F_{\hat{c}}(u, w), \quad A_{uv} = G_{uv}, \quad B_{vw} = G_{vw}.$$

The resulting instance contains a triangle if and only if G does. A masked edge of any original triangle can be placed on $U \times W$, with its third vertex in V . Conversely, all three derived edges are original edges, and the zero diagonal rules out repeated original labels.

Each derived oracle call, even in superposition over relation labels, uses at most one call to O_G . Reversibly compute the original pair for an A or B query, or for an R query with mask value one. For an R query with mask value zero, compute the pair $(1, 1)$ instead. Apply O_G to this address and the answer register, then uncompute the address. Since $G_{11} = 0$, this implements the derived bit oracle exactly. All address and mask computations use already-known data.

Conditioned on any sampled data, the chosen candidate is fixed. On a positive graph, the derived instance is always in the promise domain of Lemma 2. On a negative graph it is in the domain whenever the concentration event holds. Set $\eta = 1/100$ and amplify the sparse decision algorithm to error at most $1/100$ on its promise domain, using fresh internal randomness. The whole decision procedure then has error at most $1/50$ on every graph: at most $1/100$ for concentration failure and $1/100$ for subroutine failure. Its query cost is

$$q + O\left(n^{5/4} \max\{C_2\delta \log(2 + 1/\delta), n^{-2/5}\}^{1/8}\right),$$

with the $O(\cdot)$ constant independent of δ . Both budgets are fixed caps, including on concentration failure. For $n < n_0$, query the entire graph.

3.2 Qualitative and quantitative savings

For fixed δ , the template-learning cost is $O_\delta(n)$. The preceding bound and the uniform constant in Lemma 2 give, for an absolute constant K independent of δ ,

$$\limsup_{n \rightarrow \infty} \frac{Q_{\Delta}^{\text{dec}}(n)}{n^{5/4}} \leq K[\delta \log(2 + 1/\delta)]^{1/8}.$$

Letting $\delta \downarrow 0$ proves $Q_{\Delta}^{\text{dec}}(n) = o(n^{5/4})$. This limit argument concerns the optimal query complexity at each input size; the following parameter choice supplies the quantitative algorithm.

Template height. Write $t = \log_2^* n$. For $n > 1$, unwinding the definition gives $\text{twr}(t-1) < n \leq \text{twr}(t)$. For all sufficiently large n , define

$$\log_2 \frac{1}{\delta_n} = \frac{t-3-C_3}{C_3}, \quad \text{so} \quad \lceil C_3 \log_2(1/\delta_n) + C_3 \rceil = t-3. \quad (4)$$

Then $M = \text{twr}(t-3)$ is a valid template size. Taking two logarithms of $n > \text{twr}(t-1)$ gives

$$M < \log_2 \log_2 n.$$

Moreover $C_3 \geq 1$ implies $\delta_n^{-2} \leq 4^t$. With $\eta = 1/100$, equation (3) yields

$$q \leq 8 \cdot 4^t \left(n \ln(\log_2 \log_2 n) + (\log_2 \log_2 n)^2 + \ln 200 \right) + 1 = O\left(4^{\log_2^* n} n \log \log \log n\right) = n^{1+o(1)}.$$

Here $\log_2^* n = o(\log n)$, so both $4^{\log_2^* n}$ and the remaining iterated-logarithm factor are $n^{o(1)}$. In particular, $q \leq n^{9/8}$ eventually.

Range conditions. Since $\delta_n \rightarrow 0$, eventually $\delta_n \leq \delta_0$ and thus $\rho \leq 1$. Also $\delta_n \geq 2^{-t/C_3} \geq 2^{-t} \geq n^{-2/5}$ for large n . As $C_2 \geq 1$ and $L = \log(2 + 1/\delta_n) \geq \ln 3 > 1$, we have $\rho \geq n^{-2/5}$. Thus Lemma 2 applies in its main range. For $t \geq 1$,

$$L \leq \log(2 + 2^t) \leq (t+1) \ln 2 \leq t+1.$$

Decision bound. The decision procedure therefore uses

$$q + O\left(n^{5/4} [\delta_n L]^{1/8}\right) \leq n^{9/8} + O\left(n^{5/4} 2^{-(t-3-C_3)/(8C_3)} (t+1)^{1/8}\right) = O\left(n^{5/4} 2^{-t/(8C_3)} (t+1)^{1/8}\right)$$

queries. The fixed factor $2^{(3+C_3)/(8C_3)} \leq \sqrt{2}$ is absorbed. The sampling term is also absorbed, since $2^{-t/(8C_3)} \geq n^{-1/16}$ eventually. For every fixed $0 < c' < \ln 2/(8C_3)$,

$$2^{-t/(8C_3)} (t+1)^{1/8} = \exp\left(-\frac{\ln 2}{8C_3} t + \frac{1}{8} \log(t+1)\right) = O(\exp(-c't)).$$

After handling the finitely many smaller input sizes by exhaustive querying, we have a decision procedure with error at most $1/50$ and worst-case cost

$$D(m) = O\left(m^{5/4} \exp(-c' \log_2^* m)\right) \quad (5)$$

on every m -vertex graph. The constant is independent of m . In particular $D(m) = O(m^{5/4})$; no additional baseline algorithm will be needed below.

3.3 Witness recovery

We now recover a triangle without multiplying (5) by $\log n$. Whenever a decision test is amplified, repeat the *entire* procedure independently, including fresh template samples and fresh subroutine runs, and take a majority vote. Reusing a single learned mask would not reduce its concentration-failure probability. Error at most ε costs $O(D(m) \log(1/\varepsilon))$ queries.

First test the original graph with error at most $1/100$ and return *none* if the test is negative. Otherwise maintain a current induced subgraph. Relabeling its known vertex set allows each of its

oracle queries to be simulated by one original query. If its size m is at most 20, query all its edges and output a triangle if one exists, and *none* otherwise.

For $m > 20$, partition the current vertices into four parts whose sizes differ by at most one. Deleting each part gives four children of size at most $3m/4 + 1 \leq 4m/5$. Every triangle survives in at least one child, since its three vertices cannot meet all four parts. At depth $j = 0, 1, \dots$, test the four children with individual error at most

$$\varepsilon_j = \frac{1}{100(j+1)^2}.$$

Continue with the first child reported positive; if none is reported positive, return *none*. Each test uses fresh independent randomness, so its error bound holds conditional on the previous history. There are at most four tests per depth, and

$$\Pr(\text{any test error}) \leq \frac{1}{100} + 4 \sum_{j=0}^{\infty} \varepsilon_j < \frac{9}{100} < \frac{1}{3}.$$

If no test errs, a positive instance retains a triangle until the exhaustive final check, and a negative instance is rejected initially. Only a triple verified by the final check is output. Even after a test error, the selected vertex set shrinks geometrically; every computation branch terminates and obeys the same query budget.

It remains to bound that budget, including amplification. At depth j , every tested child has at most $(4/5)^{j+1}n$ vertices, and its amplification uses $O(\log(j+2))$ repetitions. For a tested child of size $m \geq \sqrt{n}$,

$$\log_2^* m \geq \log_2^* n - 1$$

for sufficiently large n , because $\sqrt{n} \geq \log_2 n$. Equation (5) bounds its unamplified cost by $O(m^{5/4} e^{c'} \exp(-c' \log_2^* n))$. Summing over at most four tests per depth gives

$$O\left(n^{5/4} \exp(-c' \log_2^* n) \sum_{j \geq 0} (4/5)^{5j/4} \log(j+2)\right) = O\left(n^{5/4} \exp(-c' \log_2^* n)\right).$$

Tests on children smaller than $\sqrt{n}$ have total unamplified cost $O(n^{5/8})$. To see this, the first depth with such a child has parent size less than $(4/3)(\sqrt{n}+1)$, since every child has at least $3m/4 - 1$ vertices when its parent has size m . All subsequent parent sizes decrease geometrically. The total depth is $O(\log(n+3))$, so the amplification factor in this remaining part is $O(\log \log(n+3))$. Its cost is therefore $O(n^{5/8} \log \log(n+3))$, using $D(m) = O(m^{5/4})$. This is negligible compared with $n^{5/4} \exp(-c' \log_2^* n)$, because $\exp(c' \log_2^* n) = n^{o(1)}$. The initial test and final exhaustive check fit the same bound. This proves Theorem 1, assuming Lemma 2. $\square$

4 Density-offset loading

This section records the learning-graph rules and proves the loading bound needed for Lemma 2. Section 5 then constructs and analyzes the full learning graph.

4.1 Learning-graph preliminaries

We use the extended-learning-graph framework of Carlette–Laurière–Magniez (CLM) [7]. All source numbering below refers to arXiv:1609.07786v2. For a Boolean function $f : Z \rightarrow \{0, 1\}$ on a

finite domain $Z \subseteq \{0, 1\}^d$, write $Y = f^{-1}(1)$. An extended learning graph is a rooted directed acyclic graph. Its root has empty label, and an edge $e = (u, v)$ loads one fresh coordinate j , so $S(v) = S(u) \cup \{j\}$ with $j \notin S(u)$. The two nonnegative weights $w_z^b(e)$, $b \in \{0, 1\}$, depend on z only through $z_{S(v)}$. They must satisfy

$$w_x^0(e) = w_y^1(e) \quad \text{if } x \in Z \setminus Y, \ y \in Y, \ x_{S(u)} = y_{S(u)}, \ x_j \neq y_j. \quad (6)$$

For each $y \in Y$ there is a nonnegative unit flow p_y from the root to sinks whose loaded assignments certify $f(y) = 1$. The flow is zero on every edge with $w_y^1(e) = 0$. Distinct vertices may have the same label.

For a set F of expanded edges, the negative and positive complexities are respectively

$$C^0(F, x) = \sum_{e \in F} w_x^0(e), \quad C^1(F, y) = \sum_{e \in F} \frac{p_y(e)^2}{w_y^1(e)}.$$

A term with zero numerator and denominator is zero. The maxima $C^0(F)$ and $C^1(F)$ are taken over negative and positive inputs, respectively. CLM's Theorem 3.3 gives query cost $O(\sqrt{C^0 C^1})$, so $C^0 \leq T^2$ and $C^1 \leq 1$ suffice for $O(T)$ queries. This applies to promise domains as well as total functions. Constant functions need no queries.

A super-edge is a component with one common exit for its unit flows. When attached after a cache I has been loaded, its labels include I and it queries only fresh coordinates. Lower-case c^0, c^1 denote its costs at unit incoming flow. For a loading path, both weight families and the unit path flow are defined on every assignment; the expressions $c^0(z)$ and $c^1(z)$ can therefore be evaluated regardless of a local predicate's value. Superscripts 0, 1 label weight or energy types, not the value of the bit being queried.

We use the following cost rules. Their application requires valid global compatibility, flow support, and certificates, not merely a numerical cost estimate. The universal-weight condition in Section 4.3 and the checks in Section 5.5 ensure compatibility in our construction.

- *Expansion* (CLM Lemma 3.5). Replacing super-edges by their underlying components gives $C^0(F, x) = \sum_{e \in F} c^0(e, x)$ and $C^1(F, y) = \sum_{e \in F} p_y(e)^2 c^1(e, y)$. Internal copies are kept disjoint except at their prescribed attachment vertices.
- *Speciality* (CLM Lemma 3.6). Suppose a stage carries total flow one and, on each positive input, uses exactly n_{used} of its n_{total} transitions, uniformly. Let $\sigma = n_{\text{total}}/n_{\text{used}}$, and choose input-independent bounds $P_e > 0$ for the unit positive complexities of the transitions. Multiplying every weight inside transition e by P_e/n_{used} gives positive complexity at most one and negative complexity at most

$$\sigma \mathbb{E}_e[c^0(e, x)P_e],$$

where the expectation is uniform over all transitions. One may use $P_e = c^1(e)$ when that maximum is positive, or any positive upper bound. Thus the rule does not require division by a zero complexity.

- *Dense loading* (CLM Lemma 3.7). For $N = |S| > 0$, a path loading S with weight N on each edge, for both weight families, has $c^0 = N^2$ and $c^1 = 1$.
- *OR* (CLM Lemma 4.1, with compatibility checked as above). Let G_i compute f_i on a common domain, and suppose every positive input of $f = \bigvee_{i=1}^m f_i$ has at least $k \geq 1$ successful components. Choose input-independent positive bounds $P_i \geq C^1(G_i)$. Identify the component

roots, whose labels agree, and scale all weights in G_i by P_i/k . Route flow $1/k$ into each of any k successful components. Then

$$C^1(G, y) \leq 1, \quad C^0(G, x) \leq \frac{1}{k} \sum_{i=1}^m C^0(G_i, x) P_i = \frac{m}{k} \mathbb{E}_i[C^0(G_i, x) P_i].$$

The negative bound is for $f(x) = 0$. For $k = 1$ it is a sum; for $k = m$ it is an average. Root identification implements the attachment without an artificial edge querying no coordinate.

- *Assignment-indexed construction* (CLM Theorem 4.2, construction only). At a node A with loaded cache $I(A)$, attach a separate copy $G_{A,\lambda}$ for each assignment λ to that cache, with labels enlarged by $I(A)$. Multiply its weights by $\mathbf{1}[z_{I(A)} = \lambda]$ and route flow only into the consistent copy. On each fixed input, only one copy at A has nonzero weights, so summing over assignments introduces no multiplicative cost. The component computes the corresponding restricted predicate. We use this construction, not the theorem's aggregate cost bound. In particular, we do not assume that the outer cache itself contains a triangle certificate: the terminal component supplies the missing coordinates and certifies success. Compatibility is checked in Section 5.5 and costs are computed stage by stage. Conditional sparse loading already appears in CLM's proof of Theorem 5.2 (Appendix B.1), for the marks $\{w\} \times B'$.
- *Component stages* (the final-stage normalization in CLM Theorem 4.2). A terminal stage may attach a many-sink component G_A at each of n_{total} nodes. Suppose every positive flow enters exactly n_{used} successful components, each with flow $1/n_{\text{used}}$, and their accepting labels certify the global predicate. With positive bounds P_A for their unit positive complexities, scale the weights in G_A by P_A/n_{used} . The stage has positive complexity at most one and negative complexity at most

$$\frac{n_{\text{total}}}{n_{\text{used}}} \mathbb{E}_A[C^0(G_A, x) P_A].$$

Indeed, each used component contributes at most $(1/n_{\text{used}})^2 P_A / (P_A/n_{\text{used}}) = 1/n_{\text{used}}$ to positive complexity; the negative weights add. For assignment-indexed components, take P_A uniform over the assignments at A . This is the same speciality calculation, but does not require the components to be super-edges.

An empty load is implemented by identifying its entrance and exit, with zero cost of both types. Components with no successful input can be omitted when they are never used by positive flow; a branch whose current label already certifies success can terminate there. No zero-cost attachment is implemented as an extra query edge. Costs of successive stages add; when only a fixed number of stages are composed, their positive bounds contribute only a constant factor.

4.2 The density-offset loading bound

Lemma 3. *Fix $0 < \delta \leq 1$ and a set S of $N \geq 1$ fresh input coordinates. There is an extended-learning-graph super-edge loading S whose unit-flow costs, evaluated on every assignment z , satisfy*

$$c^1(z) \leq 1, \\ c^0(z) \leq CN(|z_S| + \delta N + 1) \log(2 + 1/\delta),$$

where $|z_S|$ is the number of ones on S and one may take $C = 8$. In particular, for a fixed input and any distribution over sets S of the same size N , if $\mathbb{E}|z_S| \leq \delta N$, then

$$\mathbb{E}c^0(z) \leq 16(\delta N^2 + N) \log(2 + 1/\delta).$$

No sparsity assumption is needed for the positive bound.

Proof. Modify CLM's loading path (Lemma 3.8) as follows. Fix an order $S = (j_1, \dots, j_N)$, independent of the input, and set

$$K = \max\{1, \delta N\}, \quad L_N = 3 + \ln(1 + N/K).$$

At the edge e_i loading j_i , let $k_i = \sum_{h < i} z_{j_h}$ count the ones already loaded along this path. For $b \in \{0, 1\}$ define

$$w_z^b(e_i) = \begin{cases} L_N(K + k_i), & z_{j_i} = b, \\ L_N N, & z_{j_i} = 1 - b. \end{cases}$$

The weights are strictly positive and depend only on the path prefix and the current bit. If two assignments agree on the prefix and differ at j_i , their k_i agree and their required w^0, w^1 weights coincide. Thus (6) holds independently of the local predicate or the signs assigned to those inputs. The unique path carries unit flow on every assignment.

Put $t = |z_S|$. On the one-bits the successive prefix counts are $0, \dots, t-1$, whereas every zero-bit has positive weight $L_N N$. Consequently

$$c^1(z) = \frac{N-t}{L_N N} + \frac{1}{L_N} \sum_{k=0}^{t-1} \frac{1}{K+k}.$$

For $t \geq 1$, monotonicity of $1/(K+x)$ gives

$$\sum_{k=0}^{t-1} \frac{1}{K+k} \leq \frac{1}{K} + \int_0^{t-1} \frac{dx}{K+x} \leq \frac{1}{K} + \ln(1 + N/K).$$

For $t = 0$ the sum is empty. In both cases, since $K \geq 1$,

$$c^1(z) \leq \frac{1 + 1/K + \ln(1 + N/K)}{3 + \ln(1 + N/K)} \leq 1.$$

This argument allows nonintegral K .

For the negative cost, each one-bit has weight $L_N N$, and every prefix count on a zero-bit is at most t . Therefore

$$c^0(z) = L_N \left(Nt + \sum_{i: z_{j_i}=0} (K + k_i) \right) \leq L_N (Nt + (N-t)(K+t)) \leq L_N N(K+2t).$$

Let $L = \ln(2 + 1/\delta)$. Since $N/K \leq 1/\delta$ and $L \geq \ln 3 > 1$,

$$L_N \leq 3 + \ln(1 + 1/\delta) \leq 4L, \quad K + 2t \leq 2(t + \delta N + 1).$$

Combining these inequalities proves the bound with $C = 8$. Taking expectations over a fixed-size random S gives $8N(2\delta N + 1)L \leq 16(\delta N^2 + N)L$. No independence among the loaded bits or per-set sparsity bound is used. $\square$

4.3 Universal weights and compatibility

Definition (Universal weights). An expanded edge $e = (u, v)$ loading a fresh coordinate j has *universal weights* if

$$w_z^b(e) = g_e(z_{S(u)}, z_j \oplus b), \quad b \in \{0, 1\},$$

for a fixed nonnegative function $g_e : \{0, 1\}^{S(u)} \times \{0, 1\} \rightarrow \mathbb{R}_{\geq 0}$. Apart from its displayed arguments, the function may depend only on known labels and fixed parameters; it has no access to other input bits or to a predicate value.

These weights depend only on the head's loaded assignment. For any x, y agreeing on the tail and differing at j ,

$$w_x^0(e) = g_e(x_{S(u)}, x_j) = g_e(y_{S(u)}, y_j \oplus 1) = w_y^1(e).$$

Thus compatibility holds for every such pair, regardless of its local or global predicate values. In particular, a globally positive input that is locally negative in a component causes no compatibility problem.

Multiplication of both weight families by the same nonnegative function of the tail assignment preserves universality: replace $g_e(a, h)$ by the product of that function at a and $g_e(a, h)$. This includes constant rescalings, assignment selectors, and candidate selectors determined by the cache. Such multiplication preserves compatibility, not automatically a valid flow: where a multiplier vanishes, the positive flow must also vanish. Flow routing and accepting certificates are checked separately in Section 5.

Dense loads are universal because their weights are constant. For the loader in Lemma 3, take $g_{e_i}(a, 0) = L_N(K + k_i(a))$ and $g_{e_i}(a, 1) = L_N N$. The count $k_i(a)$ uses only already-loaded path coordinates. This proves universality for every expanded loader edge, including when an additional outer cache is present.

5 Proof of the sparse-relation lemma

We prove Lemma 2, using the relations R, B, A and the parameters δ, L, ρ from Section 2.1. The proof decomposes the triangle predicate, constructs both branches, checks compatibility, and then bounds the negative complexity by averaging over all stages. All positive-complexity bounds are uniform.

5.1 Predicate decomposition and certificate coverage

For an x -subset $X \subseteq W$ define

$$C(u, v) = \{w : R_{uw} = B_{vw} = 1\}, \quad \Delta_X = \{(u, v) : C(u, v) \cap X = \emptyset\}.$$

Let h_X detect a triangle whose W vertex belongs to X , and let f_X detect a triangle with $(u, v) \in \Delta_X$. On every input,

$$\text{Triangle} = h_X \vee f_X. \tag{7}$$

Indeed, suppose a triangle has its edge (u, v) outside Δ_X . Then a common neighbor in X , together with $A_{uv} = 1$, gives an h_X triangle.

For each X , first combine the graphs for h_X and f_X by the OR rule with one successful component. Normalize this combined graph to have positive complexity at most one. There are $\binom{n}{x}$ such graphs, all computing the same triangle predicate. Applying the OR rule with $k = \binom{n}{x}$ therefore averages their negative complexities rather than summing them. Positive flow is uniform

over X and then uses a successful h_X or f_X branch. Flow may depend on the full positive input; conductances below use only loaded answers and the current queried bit, as permitted by Section 4.

5.2 The outer rectangular stages

For f_X , choose subsets $P_0 \subset U$ and $Q_0 \subset V$ of sizes $r - 1$ and $s - 1$. Load R on $P_0 \times X$ with Lemma 3 and load B on $Q_0 \times X$ densely. Then insert one U vertex u and one V vertex v , in that order, densely loading $R_{u,X}$ and then $B_{v,X}$. Nodes are merged by $P = P_0 \cup \{u\}$ and then by $Q = Q_0 \cup \{v\}$. The data now reveal $\Delta_X \cap (P \times Q)$ exactly. Only the initial R block and the inner R marks of Section 5.3 use Lemma 3. All other loads in this proof are dense.

Fix an input with $f_X = 1$ and a residual triangle (u_*, v_*, w_*) , with $(u_*, v_*) \in \Delta_X$, so that $w_* \notin X$. Its positive flow is uniform over $P_0 \not\ni u_*$ and $Q_0 \not\ni v_*$. The insertions are u_* and then v_* , and the terminal component is entered at (P, Q) . For this fixed witness, each used P has the unique predecessor $P \setminus \{u_*\}$, and each used Q has the unique predecessor $Q \setminus \{v_*\}$. Thus the merges preserve uniform flow and flow conservation. Counting all transitions and dividing by the uniformly used ones gives the exact specialities in Table 1. For $r, s \leq n/2$ these are $O(1)$, $O(n)$, n^2/r and $n^2/(rs)$, with absolute constants. These are direct counts. No root-mean-square quantum-walk update theorem is assumed. Stages O0–O2 consist of super edges and are normalized with CLM’s speciality lemma. Stage O3 plugs the terminal component, which has many sinks, at each node (P, Q) . It is normalized as a component stage, exactly as in the final stage of CLM’s Theorem 4.2, and this gives the same formula with speciality $n^2/(rs)$. The two consecutive loads in O0 have total unit positive complexity at most two; this constant is included when normalizing that stage.

stage	all transitions	speciality	loads
O0: choose P_0, Q_0	$\binom{n}{r-1} \binom{n}{s-1}$	$\frac{n^2}{(n-r+1)(n-s+1)}$	Lemma 3 on $R_{P_0 \times X}$; dense $B_{Q_0 \times X}$
O1: insert u	$\binom{n}{r-1} \binom{n}{s-1} (n-r+1)$	$\frac{n^2}{n-s+1}$	dense $R_{u,X}$
O2: insert v	$\binom{n}{r} \binom{n}{s-1} (n-s+1)$	$\frac{n^2}{r}$	dense $B_{v,X}$
O3: check at (P, Q)	$\binom{n}{r} \binom{n}{s}$	$\frac{n^2}{rs}$	component $\bigvee_{w \in W} G_{P,Q,w}$

Table 1: Outer stages of f_X . In every stage the positive flow uses exactly $\binom{n-1}{r-1} \binom{n-1}{s-1}$ transitions, uniformly; the speciality is the ratio.

5.3 The modified terminal check

The terminal component at (P, Q) is an OR over $w \in W$ of components $G_{P,Q,w}$. For $w \in X$ the local predicate is identically false: an enabled candidate would need $w \in C(u, v) \cap X$, which contradicts $(u, v) \in \Delta_X$. These branches are empty and use no queries. For $w \notin X$, the component $G_{P,Q,w}$ has four stages. Here $b_1 \geq 1$ is an integer parameter. Empty loads, including I0 when $b_1 = 1$, are contracted as specified in Section 4.

- I0. Choose $P'_0 \subset P$ with $|P'_0| = b_1 - 1$, and load the marks $R_{p,w}$, $p \in P'_0$, with Lemma 3 ($N = b_1 - 1$, same δ).
- I1. Insert $u \in P \setminus P'_0$ and load the single bit R_{uw} . Nodes are then merged by $P' = P'_0 \cup \{u\}$, a b_1 -subset of P .

- I2. Choose a single $v \in Q$ and load the single bit B_{vw} . Thus the inner V -side subset has size $b_2 = 1$.
- I3. Take the OR of A_{uv} over the candidate list

$$D = D(P', v, w) = \{(u, v) : u \in P', (u, v) \in \Delta_X, R_{uw} = B_{vw} = 1\}.$$

The list D is determined by loaded bits. Membership in Δ_X uses $R_{u,X}$ and $B_{v,X}$, which lie in the outer caches because $u \in P$ and $v \in Q$. The marks R_{uw} come from I0 or I1, and B_{vw} comes from I2. Every inner coordinate is fresh: the outer caches contain only coordinates $R_{\cdot,X}$ and $B_{\cdot,X}$, while $w \notin X$; I1 excludes the vertices already chosen in I0, and no A coordinate has previously been queried.

Positive flow for the residual triangle goes to $w = w_*$. It is then uniform over $P'_0 \not\ni u_*$, followed by $u = u_*$, $v = v_*$ and the candidate (u_*, v_*) . This candidate is enabled, because $(u_*, v_*) \in \Delta_X$ and $R_{u_*w_*} = B_{v_*w_*} = 1$. Each used P' has the unique predecessor $P' \setminus \{u_*\}$, so merging preserves equal flow on the used transitions. The accepting sink contains the three one-bits $R_{u_*w_*}$, $B_{v_*w_*}$ and $A_{u_*v_*}$, certifying a triangle. It also contains $R_{u_*,X}$ and $B_{v_*,X}$, which certify $(u_*, v_*) \in \Delta_X$. Thus it certifies the local predicate f_X as well as the global triangle predicate. The exact specialities, again direct counts, are shown in Table 2. For $b_1 \leq r/2$ they are at most 2, r , rs/b_1 and rs/b_1 . Stages I0–I2 consist of (super) edges and use the speciality lemma. Stage I3 plugs the OR over D , which has many sinks, at each node (P', v) , and is normalized as a component stage (final stage of CLM's Theorem 4.2). The final OR has weight one on each enabled candidate query and weight zero on disabled ones. It has negative complexity $|D|$ and positive complexity one on a positive local input, by routing unit flow through an enabled candidate with $A_{uv} = 1$. If D is empty, the component is false and receives no positive flow.

stage	all transitions	speciality	loads
I0: choose P'_0	$\binom{r}{b_1-1}$	$\frac{r}{r-b_1+1}$	Lemma 3 on $R_{P'_0 \times \{w\}}$
I1: insert u	$\binom{r}{b_1-1}(r-b_1+1)$	r	R_{uw}
I2: choose v	$\binom{r}{b_1} s$	$\frac{rs}{b_1}$	B_{vw}
I3: check at (P', v)	$\binom{r}{b_1} s$	$\frac{rs}{b_1}$	OR of A_{uv} , $(u, v) \in D$

Table 2: Inner stages of $G_{P,Q,w}$, $w \notin X$. In every stage the positive flow uses exactly $\binom{r-1}{b_1-1}$ transitions, uniformly.

The choice $b_2 = 1$ avoids a dense initial load on the V side without increasing the averaged check cost. Inserting u before v gives the smaller speciality r for the R_{uw} query, as recorded in Table 2.

5.4 The h_X branch

Known-graph collision asks whether a known simple graph has an edge whose endpoints are both marked one; its input consists of the unknown Boolean vertex marks. We use the constant-weight subset construction of [7, 12]. On N vertices, choose an integer $2 \leq k \leq N/2$, densely load the marks of a $(k-2)$ -subset, and then insert two distinct vertices in order, querying their marks and merging by the enlarged subsets at each stage. For a fixed marked edge with ordered endpoints, the

positive flow is uniform over initial subsets avoiding both endpoints, and inserts those endpoints. Each stage uses exactly $\binom{N-2}{k-2}$ transitions. The exact specialities are

$$\frac{N(N-1)}{(N-k+2)(N-k+1)}, \quad \frac{N(N-1)}{N-k+1}, \quad \frac{N(N-1)}{k-1}.$$

Consequently the normalized stage costs are $O(k^2)$, $O(N)$ and $O(N^2/k)$, with total positive complexity at most three. Taking $k = \lceil N^{2/3} \rceil$ gives negative complexity $O(N^{4/3})$; bounded values of N for which $k \leq N/2$ fails are handled by loading all marks. The accepting label contains the two marked endpoints of a known edge. When the known graph depends on an outer cache, use assignment-indexed copies selected by that cache.

For an h_X witness, fix its U endpoint. Densely load R on $Y_0 \times X$ for a $(y-1)$ -subset $Y_0 \subset U$, and then insert one U vertex and densely load its row on X . Positive flow chooses Y_0 uniformly avoiding the fixed endpoint and then inserts it, giving Y of size y . These loaded R bits give the known bipartite graph on $X \cup Y$. For each $v \in V$, the unknown marks are B_{vx} on X and A_{uv} on Y . A known-graph collision is exactly the desired triangle. Its three one-bits are all loaded, including its known R edge. The outer specialities are exactly $n/(n-y+1)$, n and n/y . The OR over v has positive complexity at most one and negative complexity at most three times the sum of the collision costs. The last outer stage plugs the OR over v of collision graphs, which has many sinks, at each loaded node; it is normalized as a component stage (final stage of CLM's Theorem 4.2). Thus this branch has positive complexity at most three. Writing H_X for its learning-graph complexity, its negative complexity and H_X^2 are both bounded by the following expression:

$$H_X^2 = O\left(x^2y^2 + nx^2 + \frac{n^2(x+y)^{4/3}}{y}\right), \quad \text{and for } y = x : \quad H_X^2 = O(x^4 + nx^2 + n^2x^{1/3}).$$

No conversion of an arbitrary quantum algorithm into a learning graph is being asserted.

5.5 Compatibility inside assignment-indexed components

These conditional components use CLM's assignment-indexed construction (Theorem 4.2 of [7], construction only). We verify its compatibility here directly, through universality, rather than importing it. Let I be the outer cache loaded before the terminal component. For each assignment λ to I there is one copy of the terminal component, with every weight multiplied by $\mathbf{1}[z_I = \lambda]$. Inconsistent copies get zero weight and zero flow. This factor is a function of the tail label of every inner edge, since $I \subseteq S(u)$ there. The inner loader's prefix consists of the outer cache and its own earlier bits. Its weights use only its own count k and the fixed numbers $N = b_1 - 1$, K and L_N . The candidate edge loading A_{uv} has weight $\mathbf{1}[(u, v) \in D]$. This is a function of the tail label, independent of the new bit and of b . The rescalings in CLM's speciality and OR lemmas, and in the component-stage normalization, are constants, because they use the uniform positive-complexity bounds of the components. By the definition of universal weights in Section 4.3, every weight in the global graph is therefore universal. Compatibility holds for every (global negative, global positive) pair, including positive inputs that are locally negative in some component. Agreement on the loaded prefix implies agreement on the component selector and on D . Labels are sets and may repeat across vertices. Super edges are expanded as disjoint copies, so different sets P'_0 with shared prefixes do not conflict. On a fixed input only the consistent assignment copy is active, so summing weights over copies introduces no factor for their number. The prescribed positive flows use only consistent copies and enabled candidates, and all remaining loaded edges have strictly positive weights. These flows therefore vanish wherever the positive weight is zero. The witness routing

above conserves flow and terminates at certificates. These facts check the flow requirements as well as compatibility. The construction does not assume that an unknown input-dependent graph is available for free.

5.6 Averaging identities and the master cost inequality

We use the mixed common-neighbor averaging bound from CLM's triangle algorithm (Lemma A.1), originating in Le Gall's decomposition [8]. For X a uniform x -subset of a universe of size n and a fixed set C ,

$$\frac{|C|}{n} \Pr(X \cap C = \emptyset) = O(1/x). \quad (8)$$

For completeness, if $a = |C|$ and $1 \leq x \leq n - a$, then

$$\Pr(X \cap C = \emptyset) = \frac{\binom{n-a}{x}}{\binom{n}{x}} \leq (1 - a/n)^x \leq e^{-ax/n}.$$

The probability is zero when $x > n - a$. For $a = 0$ the left side of (8) is zero; for $a > 0$ it is at most $1/(ex)$, since $te^{-xt} \leq 1/(ex)$ for $t \geq 0$. The bound depends only on $|C|$, so it applies unchanged to the common-neighbor sets defined using the two different relations R and B .

Fix a negative input z in the promise domain, so $|R| \leq \delta n^2$. Every stage has positive complexity at most one after normalization, and each level has at most four stages. Hence all positive complexities are bounded by absolute constants, and they only affect constants below.

Inner component. By the speciality lemma for I0–I2, the component-stage normalization for I3, and Lemma 3, for $w \notin X$ and $b_1 \leq r/2$,

$$\begin{aligned} C^0(G_{P,Q,w}, z) &\leq 2 \mathbb{E}_{P'_0} [c_{\text{load}}^0] + r + \frac{rs}{b_1} + \frac{rs}{b_1} \mathbb{E}_{P',v} |D|, \\ c_{\text{load}}^0 &\leq C(b_1 - 1)(|R \cap (P'_0 \times \{w\})| + \delta(b_1 - 1) + 1)L. \end{aligned}$$

OR over w and outer check. By CLM's OR lemma with $k = 1$, the OR over w is a *sum*; the factor 4 below bounds $C^1(G_{P,Q,w})$, since each of its four stages has positive complexity at most one. The outer check stage, a component stage, multiplies by its speciality:

$$C^0(G_{P,Q}, z) \leq 4 \sum_{w \in W} C^0(G_{P,Q,w}, z), \quad \text{contribution of O3} \leq \frac{n^2}{rs} \mathbb{E}_{P,Q} C^0(G_{P,Q}, z).$$

Finally the combined $h_X \vee f_X$ graphs are averaged over X . The expectations here are over all stage transitions, not just those used by a positive flow: P, Q are independent uniform subsets; P'_0, P' are uniform subsets of their prescribed sizes inside P ; and v is uniform in Q . These sampling rules do not depend on X .

Identity 1 (loader mass). If P is a uniform r -subset of U and P'_0 is a uniform $(b_1 - 1)$ -subset of P , then P'_0 is a uniform $(b_1 - 1)$ -subset of U . Hence

$$\sum_{w \in W} \mathbb{E}_{P,P'_0} |R \cap (P'_0 \times \{w\})| = \mathbb{E} |R \cap (P'_0 \times W)| = \frac{(b_1 - 1)|R|}{n} \leq \delta(b_1 - 1)n. \quad (9)$$

The omitted branches $w \in X$ have zero cost; extending their nonnegative loader bounds to the sum over all W only increases the bound. The loader bound is linear in the number of loaded ones. Therefore no per- w sparsity is needed, and heavy vertices w are paid for on average. The equalities in (9) and (10) below are exact. The corresponding cost contributions carry, in addition, absolute constant factors from the normalization, namely the factor 4 from $C^1(G_{P,Q,w}) \leq 4$ and the factor at most 2 from the I0 speciality; these are absorbed into the $O(\cdot)$ bounds and into C_1 below. After the outer factor, the loader contributes

$$\frac{n^2}{rs} \cdot O\left(L(b_1 - 1)[\delta(b_1 - 1)n + n(\delta(b_1 - 1) + 1)]\right) = O\left(\frac{n^3 L(\delta b_1^2 + b_1)}{rs}\right).$$

Identity 2 (check term). Let P be a uniform r -subset of U , P' a uniform b_1 -subset of P , Q a uniform s -subset of V , and v a uniform element of Q . Then P' is a uniform b_1 -subset of U and v is uniform on V , independently. Fix a pair $(u_0, v_0) \in U \times V$. It is a potential candidate exactly when $u_0 \in P'$ and $v = v_0$, which has probability $(b_1/n)(1/n) = b_1/n^2$. It belongs to $D(P', v, w)$ precisely when, in addition, $(u_0, v_0) \in \Delta_X$ and $w \in C(u_0, v_0)$. Summing over the fixed pairs and over w gives $\sum_w \mathbb{E}_{P,Q,P',v} |D| = (b_1/n^2) \sum_{(u,v) \in \Delta_X} |C(u, v)|$, that is,

$$\frac{n^2}{rs} \sum_{w \in W} \mathbb{E}_{P,Q} \left[\frac{rs}{b_1} \mathbb{E}_{P',v} |D| \right] = \sum_{(u,v) \in \Delta_X} |C(u, v)|. \quad (10)$$

Averaging over X and applying (8) to each of the n^2 pairs bounds this by $n^2 \cdot O(n/x) = O(n^3/x)$. Equivalently, the average of $|D|$ over w , X and the transitions is $O(b_1/x)$. Up to the same absolute factors, the remaining inner terms contribute $(n^2/(rs)) \cdot n \cdot (r + rs/b_1) = n^3/s + n^3/b_1$.

Outer stages. On every negative input in the domain, $\mathbb{E}_{P_0,X} |R \cap (P_0 \times X)| = (r-1)x|R|/n^2 \leq \delta r x$. So stage O0 costs $O(L(\delta r^2 x^2 + rx) + s^2 x^2)$. Stages O1 and O2 cost $O(nx^2)$ and $O(n^2 x^2/r)$.

The cost inequality. The four normalized stages of the graph for f_X give positive complexity at most four; the graph for h_X has positive complexity at most three. The OR rules above combine them, and then all X , with global positive complexity at most one. Their negative cost is bounded uniformly over all negative inputs in the promise domain by the expression below. Taking its maximum over those inputs, the learning-graph complexity T therefore satisfies, for an absolute constant C_1 ,

$$T^2 \leq C_1 \left[x^2 y^2 + nx^2 + \frac{n^2(x+y)^{4/3}}{y} + L(\delta r^2 x^2 + rx) + s^2 x^2 + \frac{n^2 x^2}{r} + \frac{n^3 L \delta b_1^2}{rs} + \frac{n^3 L b_1}{rs} + \frac{n^3}{s} + \frac{n^3}{b_1} + \frac{n^3}{x} \right]. \quad (11)$$

This holds for positive integer parameters satisfying $1 \leq b_1 \leq r/2$ and $r, s, x, y \leq n/2$. Loading the I0 marks densely would instead contribute $O(n^3 b_1^2/(rs))$; the offset loader gives $O(n^3 L(\delta b_1^2 + b_1)/(rs))$. Taking the inner V -side subset to have size one leaves the separate insertion cost n^3/s , which is retained. Inequality (11) concerns global negative inputs only. Some local f_X branches can be false on a dense globally positive input, and their negative complexities need not be bounded. All local positive complexities are uniformly bounded, so this does not affect the global construction.

5.7 Parameter selection, feasibility, and the small-density floor

First assume $n^{-2/5} \leq \rho \leq 1$. Then $\delta \geq 1/n$ for $n \geq n_0$. Otherwise $\rho \leq C_2 n^{-1} \log(2+n) < n^{-2/5}$, by monotonicity. Consequently $L \leq \log(2+n)$. Up to integer rounding, choose

$$x = y = b_1 = n^{1/2} \rho^{-1/4}, \quad r = n^{3/4} \rho^{-1/8}, \quad s = n^{3/4} \rho^{3/8}. \quad (12)$$

Writing $\rho = n^{-\kappa}$ with $0 \leq \kappa \leq 2/5$, this is $r = n^{3/4+\kappa/8}$, $s = n^{3/4-3\kappa/8}$ and $x = b_1 = n^{1/2+\kappa/4}$. The parameters satisfy $b_1 = (rs/\rho)^{1/3}$, which is the balance point of $n^3 \rho b_1^2 / (rs)$ against n^3 / b_1 . Since $L\delta = \rho/C_2 \leq \rho$, the five principal terms satisfy

$$L\delta r^2 x^2 \leq \rho r^2 x^2 = s^2 x^2 = \frac{n^3 \rho b_1^2}{rs} = \frac{n^3}{b_1} = \frac{n^3}{x} = n^{5/2} \rho^{1/4} =: \Pi,$$

and $n^3 L\delta b_1^2 / (rs) \leq \Pi$ as well. Table 3 lists every term of (11) at these parameters.

term(s)	ratio to Π	uniform ratio bound
$L\delta r^2 x^2, s^2 x^2,$ $n^3 L\delta b_1^2 / (rs), n^3 / b_1, n^3 / x$	≤ 1	1
Lrx	$Ln^{-5/4} \rho^{-5/8}$	Ln^{-1}
nx^2	$n^{-1/2} \rho^{-3/4}$	$n^{-1/5}$
$n^2 x^2 / r, n^3 / s$	$n^{-1/4} \rho^{-5/8}$	1
$n^3 Lb_1 / (rs)$	$Ln^{-1/2} \rho^{-3/4}$	$Ln^{-1/5}$
$x^2 y^2 = x^4$	$n^{-1/2} \rho^{-5/4}$	1
$n^2 (x+y)^{4/3} / y$	$O(n^{-1/3} \rho^{-1/3})$	$O(n^{-1/5})$

Table 3: All terms of the cost inequality (11) at the parameters (12), where $\Pi = n^{5/2} \rho^{1/4}$, $n^{-2/5} \leq \rho \leq 1$, and $L \leq \log(2+n)$. Bounds apply separately to each term in a row.

Every term is at most an absolute constant times Π throughout $n^{-2/5} \leq \rho \leq 1$, where $L \leq \log(2+n)$. At the lower endpoint $\rho = n^{-2/5}$ three lower-order terms tie with Π : $n^2 x^2 / r$, n^3 / s and x^4 . This is harmless for an $O(\cdot)$ bound, but there is no polynomial slack at the endpoint.

Feasibility and rounding. For $n^{-2/5} \leq \rho \leq 1$,

$$\frac{r}{n} = \frac{b_1}{r} = n^{-1/4} \rho^{-1/8} \leq n^{-1/5}, \quad \frac{s}{n} \leq n^{-1/4}, \quad \frac{x}{n} = \frac{y}{n} \leq n^{-2/5}, \quad s \geq n^{3/5}, \quad b_1 \geq n^{1/2}.$$

Hence, for $n \geq n_0$ and uniformly in ρ , all subsets lie below half their universes, $b_1 \leq r/2$, and $b_2 = 1 \leq s/2$. Round each parameter up. All parameters are at least one, so rounding changes each by a factor at most 2. Every term of (11) is nondecreasing in its numerator parameters, and has total numerator degree at most 4. Rounding therefore increases costs by a factor at most 16; denominator parameters can only increase. The constraints survive uniformly after enlarging n_0 , since each displayed ratio defining an upper-size constraint is multiplied by at most two. Thus $T^2 \leq C'\Pi = C'n^{5/2} \rho^{1/4}$ with C' absolute. Taking the square root gives $T \leq \sqrt{C'} n^{5/4} \rho^{1/8}$. CLM's Theorem 3.3 on the promise domain gives a bounded-error algorithm with at most $Cn^{5/4} \rho^{1/8}$ queries. The cap is a function of (n, δ) only. All conductances used above are functions of loaded answers, known labels, and fixed size parameters. The resulting query algorithm has a prescribed finite query budget; running it on an oracle outside the promise domain can change its correctness, but not that budget. Thus the same cap holds on every input, as required in Lemma 2.

The floor. If $\rho < n^{-2/5}$, let $\bar{\delta} > \delta$ solve $C_2 \bar{\delta} \log(2 + 1/\bar{\delta}) = n^{-2/5}$. It exists by the monotonicity and continuity noted in Section 2.1; moreover $\bar{\delta} \leq \delta_0$, since $n^{-2/5} \leq 1$. The promise $|R| \leq \delta n^2$ implies $|R| \leq \bar{\delta} n^2$. Run the algorithm for $\bar{\delta}$. Its cap is $C n^{5/4} (n^{-2/5})^{1/8} = C n^{6/5}$. This proves Lemma 2. □

Use of AI: The author prepared a draft strategy and used ChatGPT and Claude to work out the details, iteratively verify and expand the proofs. The author is solely responsible for the correctness of the proofs.

References

- [1] H. Buhrman, C. Dürr, M. Heiligman, P. Høyer, F. Magniez, M. Santha, R. de Wolf. *Quantum Algorithms for Element Distinctness*. SIAM Journal on Computing 34(6) (2005), 1324–1330.
- [2] M. Szegedy. *On the Quantum Query Complexity of Detecting Triangles in Graphs*. [arXiv:quant-ph/0310107](https://arxiv.org/abs/quant-ph/0310107).
- [3] F. Magniez, M. Santha, M. Szegedy. *Quantum Algorithms for the Triangle Problem*. SIAM Journal on Computing 37(2) (2007), 413–424.
- [4] T. Lee, F. Magniez, M. Santha. *Improved Quantum Query Algorithms for Triangle Finding and Associativity Testing*. [arXiv:1210.1014](https://arxiv.org/abs/1210.1014).
- [5] A. Belovs, A. Rosmanis. *On the Power of Non-Adaptive Learning Graphs*. [arXiv:1210.3279](https://arxiv.org/abs/1210.3279).
- [6] S. Jeffery, R. Kothari, F. Magniez. *Nested Quantum Walks with Quantum Data Structures*. Proceedings of SODA 2013, 1474–1485.
- [7] T. Carette, M. Laurière, F. Magniez. *Extended Learning Graphs for Triangle Finding*. STACS 2017, LIPIcs 66, Article 20; [arXiv:1609.07786](https://arxiv.org/abs/1609.07786).
- [8] F. Le Gall. *Improved Quantum Algorithm for Triangle Finding via Combinatorial Arguments*. FOCS 2014; [arXiv:1407.0085](https://arxiv.org/abs/1407.0085).
- [9] F. Le Gall, S. Nakajima. *Quantum Algorithm for Triangle Finding in Sparse Graphs*. Algorithmica 79 (2017), 941–959; [arXiv:1507.06878](https://arxiv.org/abs/1507.06878).
- [10] G. Li, L. Li. *Derandomization of Quantum Algorithm for Triangle Finding*. Information and Computation 304 (2025), Article 105295; [arXiv:2309.13268](https://arxiv.org/abs/2309.13268).
- [11] A. S. Gilani, D. Wang, P. Wu, X. Zhou. *Quantum Algorithms on Edge Lists: Hiding, Shuffling, and Cycle Finding*. ICALP 2026, LIPIcs 374, 97:1–97:16; [doi:10.4230/LIPIcs.ICALP.2026.97](https://doi.org/10.4230/LIPIcs.ICALP.2026.97); [arXiv:2412.17786](https://arxiv.org/abs/2412.17786).
- [12] A. Belovs. *Span Programs for Functions with Constant-Sized 1-Certificates*. STOC 2012; [arXiv:1105.4024](https://arxiv.org/abs/1105.4024).
- [13] J. Fox, Y. Zhao. *Removal lemmas and approximate homomorphisms*. Combinatorics, Probability and Computing 31 (2022), 721–736. [arXiv:2104.11626](https://arxiv.org/abs/2104.11626), especially Theorems 1.6 and 1.15, inequality (1.1) and Section 5.1.
- [14] J. Fox. *A new proof of the graph removal lemma*. Annals of Mathematics 174 (2011), 561–579. [arXiv:1006.1300](https://arxiv.org/abs/1006.1300), especially Theorem 1.
- [15] C. Hoppen, Y. Kohayakawa, R. Lang, H. Lefmann, H. Stagni. *Estimating parameters associated with monotone properties*. Combinatorics, Probability and Computing 29 (2020), 616–632, especially Section 3.3.
- [16] O. Goldreich, S. Goldwasser, D. Ron. *Property Testing and Its Connection to Learning and Approximation*. Journal of the ACM 45(4) (1998), 653–750. [Full text](#).